AFRILANG Stdlib

std/c/nlplevenshtein

Français. Bibliothèque AFRILANG ‘std/c/nlplevenshtein‘ (niveau complex, catégorie complex). Elle expose 6 fonction(s) : distance, similarity, ratio, isClose, normalizedDist, longestCommon.
‘import ”std/c/nlplevenshtein”‘ · ‘use nlplevenshtein‘
- ‘distance(a text, b text) → number‘ - ‘similarity(a text, b text) → number‘ - ‘ratio(a text, b text) → number‘ - ‘isClose(a text, b text, maxDist number) → bool‘ - ‘normalizedDist(a text, b text) → number‘ - ‘longestCommon(a text, b text) → number‘ **English.** AFRILANG library ‘std/c/nlplevenshtein‘ (tier complex, category complex). Exports 6 function(s): distance, similarity, ratio, isClose, normalizedDist, longestCommon.
‘import ”std/c/nlplevenshtein”‘ · ‘use nlplevenshtein‘
- ‘distance(a text, b text) → number‘ - ‘similarity(a text, b text) → number‘ - ‘ratio(a text, b text) → number‘ - ‘isClose(a text, b text, maxDist number) → bool‘ - ‘normalizedDist(a text, b text) → number‘ - ‘longestCommon(a text, b text) → number‘

Catégorie / Category	Modules complex (c/)	
Niveau / Tier	complex	June 16, 2026
Fonctions / Functions	6	

Contents

Français

1. Vue d’ensemble

Bibliothèque AFRILANG ‘std/c/nlplevenshtein‘ (niveau complex, catégorie complex). Elle expose 6 fonction(s) : distance, similarity, ratio, isClose, normalizedDist, longestCommon.

‘import ”std/c/nlplevenshtein”‘ · ‘use nlplevenshtein‘

```
- `distance(a text, b text) → number`
- `similarity(a text, b text) → number`
- `ratio(a text, b text) → number`
- `isClose(a text, b text, maxDist number) → bool`
- `normalizedDist(a text, b text) → number`
- `longestCommon(a text, b text) → number`
```

2. Installation & import

Ajoutez le module à votre programme :

```
import "std/c/nlplevenshtein"
use nlplevenshtein
```

```
Niveau : complex · Catégorie : Modules complex (c/)
Domaines avancés – graphes, NLP, réseaux complexes.
```

3. Référence API

```
•
 distance(a text, b text) → number•
 similarity(a text, b text) → number•
 ratio(a text, b text) → number•
 isClose(a text, b text, maxDist number) → bool•
 normalizedDist(a text, b text) → number•
 longestCommon(a text, b text) → number
```

4. Exemples d’utilisation

```
import "std/c/nlplevenshtein"
use nlplevenshtein
```

```
create result = distance(/* args */)
say result
```

```
create result = similarity(/* args */)
say result
```

```
create result = ratio(/* args */)
say result
```

```
create result = isClose(/* args */)
say result
```

```
create result = normalizedDist(/* args */)
say result
```

```
create result = longestCommon(/* args */)
say result
```

5. Bonnes pratiques

```
•
 Préférez `import "std/c/nlplevenshtein"` en tête de fichier.•
 Lancez `afrilang check` avant de livrer.•
 Combinez avec les modules stdlib de la même catégorie.•
 Voir /stdlib/ pour le source live et les démos playground.•
 Signalez les problèmes sur GitHub si une export manque.
```

6. Annexe — source

module nlplevenshtein

```
function distance(a text, b text) returns number
end
```

```
function similarity(a text, b text) returns number
end
```

```
function ratio(a text, b text) returns number
end
```

```
function isClose(a text, b text, maxDist number) returns bool
end
```

```
function normalizedDist(a text, b text) returns number
end
```

```
function longestCommon(a text, b text) returns number
end
end
```

English

1. Overview

AFRILANG library ‘std/c/nlplevenshtein‘ (tier complex, category complex). Exports 6 function(s): distance, similarity, ratio, isClose, normalizedDist, longestCommon.

‘import ”std/c/nlplevenshtein”‘ · ‘use nlplevenshtein‘

```
- `distance(a text, b text) → number`
- `similarity(a text, b text) → number`
- `ratio(a text, b text) → number`
- `isClose(a text, b text, maxDist number) → bool`
- `normalizedDist(a text, b text) → number`
- `longestCommon(a text, b text) → number`
```

2. Installation & import

Add the module to your program:

```
import "std/c/nlplevenshtein"
use nlplevenshtein
```

```
Tier: complex · Category: Complex modules (c/)
Advanced domains — graphs, NLP, complex networks.
```

3. API reference

```
•
 distance(a text, b text) → number•
 similarity(a text, b text) → number•
 ratio(a text, b text) → number•
 isClose(a text, b text, maxDist number) → bool•
 normalizedDist(a text, b text) → number•
 longestCommon(a text, b text) → number
```

4. Usage examples

```
import "std/c/nlplevenshtein"
use nlplevenshtein
```

```
create result = distance(/* args */)
say result
```

```
create result = similarity(/* args */)
say result
```

```
create result = ratio(/* args */)
say result
```

```
create result = isClose(/* args */)
say result
```

```
create result = normalizedDist(/* args */)
say result
```

```
create result = longestCommon(/* args */)
say result
```

5. Best practices

```
•
 Prefer `import "std/c/nlplevenshtein"` at the top of your file.•
 Run `afrilang check` before shipping.•
 Combine with related stdlib modules from the same category.•
 See /stdlib/ for the live source and playground demos.•
 Report issues on GitHub if an export is missing or undocumented.
```

6. Appendix — source

module nlplevenshtein

```
function distance(a text, b text) returns number
end
```

```
function similarity(a text, b text) returns number
end
```

```
function ratio(a text, b text) returns number
end
```

```
function isClose(a text, b text, maxDist number) returns bool
end
```

```
function normalizedDist(a text, b text) returns number
end
```

```
function longestCommon(a text, b text) returns number
end
end
```

Référence rapide / Quick reference

- Import : `import "std/c/nlplevenshtein"`
- Vérification : `afrilang check`
- Documentation : https://afrilang.dev/stdlib/std/c/nlplevenshtein/

Source (extrait)

```
module nlplevenshtein

function distance(a text, b text) returns number
end

function similarity(a text, b text) returns number
end

function ratio(a text, b text) returns number
end

function isClose(a text, b text, maxDist number) returns bool
end

function normalizedDist(a text, b text) returns number
```

```
end

function longestCommon(a text, b text) returns number
end
end
```